NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System

Xi Victoria Lin*, Chenglong Wang, Luke Zettlemoyer, Michael D. Ernst
Salesforce Research, University of Washington, University of Washington, University of Washington
xilin@salesforce.com, {clwang,lsz,mernst}@cs.washington.edu

Abstract

We present new data and semantic parsing methods for the problem of mapping English sentences to Bash commands (NL2Bash). Our long-term goal is to enable any user to perform operations such as file manipulation, search, and application-specific scripting by simply stating their goals in English. We take a first step in this domain, by providing a new dataset of challenging but commonly used Bash commands and expert-written English descriptions, along with baseline methods to establish performance levels on this task.

Keywords: Natural Language Programming, Natural Language Interface, Semantic Parsing

1. Introduction

The dream of using English or any other natural language to program computers has existed for almost as long as the task of programming itself (Sammet, 1966). Although significantly less precise than a formal language (Dijkstra, 1978), natural language as a programming medium would be universally accessible and would support the automation of highly repetitive tasks such as file manipulation, search, and application-specific scripting (Wilensky et al., 1984; Wilensky et al., 1988; Dahl et al., 1994; Quirk et al., 2015; Desai et al., 2016).

This work presents new data and semantic parsing methods on a novel and ambitious domain — natural language control of the operating system. Our long-term goal is to enable any user to perform tasks on their computers by simply stating their goals in natural language (NL). We take a first step in this direction, by providing a large new dataset (NL2Bash) of challenging but commonly used commands and expert-written descriptions, along with baseline methods to establish performance levels on this task.

The NL2Bash problem can be seen as a type of semantic parsing, where the goal is to map sentences to formal representations of their underlying meaning (Mooney, 2014). The dataset we introduce provides a new type of target meaning representations (Bash[1] commands), and is significantly larger (from two to ten times) than most existing semantic parsing benchmarks (Dahl et al., 1994; Popescu et al., 2003). Other recent work in semantic parsing has also focused on programming languages, including regular expressions (Locascio et al., 2016), IFTTT scripts (Quirk et al., 2015), and SQL queries (Kwiatkowski et al., 2013; Iyer et al., 2017; Zhong et al., 2017). However, the shell command data we consider raises unique challenges, due to its irregular syntax (no syntax tree representation for the command options), wide domain coverage ($>$ 100 Bash utilities), and a large percentage of unseen words (e.g. commands can manipulate arbitrary files).

We constructed the NL2Bash corpus with frequently used Bash commands scraped from websites such as question-answering forums, tutorials, tech blogs, and course materials. We gathered a set of high-quality descriptions of the commands from Bash programmers. Table 1 shows several examples. After careful quality control, we were able to gather over 9,000 English-command pairs, covering over 100 unique Bash utilities.

We also present a set of experiments to demonstrate that NL2Bash is a challenging task which is worthy of future study. We build on recent work in neural semantic parsing (Dong and Lapata, 2016; Ling et al., 2016), by evaluating the standard Seq2seq model (Sutskever et al., 2014) and the CopyNet model (Gu et al., 2016). We also include a recently proposed stage-wise neural semantic parsing model, Tellina, which uses manually defined heuristics for better detecting and translating the command arguments (Lin et al., 2017). We found that when applied at the right sequence granularity (sub-tokens), CopyNet significantly outperforms the stage-wise model, with significantly less pre-processing and post-processing. Our best performing system obtains top-1 command structure accuracy of 49%, and top-1 full command accuracy of 36%. These performance levels, although far from perfect, are high enough to be practically useful in a well-designed interface (Lin et al., 2017), and also suggest ample room for future modeling innovations.

2. Domain: Linux Shell Commands

A shell command consists of three basic components, as shown in Table 1: utility (e.g. `find`, `grep`), option flags (e.g. `-name`, `-i`), and arguments (e.g. `"*.java"`, `"TODO"`). A utility can have idiomatic syntax for flags (see the `-exec ... {} \;` option of the `find` command).

There are over 250 Bash utilities, and new ones are regularly added by third party developers. We focus on 135 of the most useful utilities identified by the Linux user group (http://www.oliverelliott.org/article/computing/ref_unix/), that is, our domain of target commands contain only those 135 utilities.[2] We only considered the target commands that can

* Work done at the University of Washington.

[1] The Bourne-again shell (Bash) is the most popular Unix shell and command language: https://www.gnu.org/software/bash/. Our data collection approach and baseline models can be trivially generalized to other command languages.

[2] We were able to gather fewer examples for the less common ones. Providing the descriptions for them also requires a higher level of Bash expertise of the corpus annotators.